

Experiment No. 3

Question:

Design a lexical Analyzer for given language should ignore the redundant spaces, tabs and new lines and ignore comments using C

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

int isKeyword(char buffer[])
{
    char keywords[32][10] =
{"main", "auto", "break", "case", "char", "const", "continue", "default",
 "do", "double", "else", "enum", "extern", "float", "for", "goto",
 "if", "int", "long", "register", "return", "short", "signed",
 "sizeof", "static", "struct", "switch", "typedef",
 "unsigned", "void", "printf", "while"};
    int i, flag = 0;
    for(i = 0; i < 32; ++i)
    {
        if(strcmp(keywords[i], buffer) == 0)
        {
            flag = 1;
            break;
        }
    }
    return flag;
}

int main()
{
    char ch, buffer[15], operators[] = "+-*/%=";
    FILE *fp;
    int i, j = 0;
    fp = stdin;
    while((ch = fgetc(fp)) != EOF)
    {
        for(i = 0; i < 6; ++i)
        {
            if(ch == operators[i])
                printf("%c is operator\n", ch);
        }
        if(isalnum(ch))
        {
            buffer[j++] = ch;
        }
        else if((ch == ' ' || ch == '\n') && (j != 0))
        {
            buffer[j] = '\0';
            j = 0;
            if(isKeyword(buffer) == 1)
                printf("%s is keyword\n", buffer);
        }
    }
}
```

```

        else
            printf("%s is identifier\n", buffer);
    }
}
return 0;
}

```

Code Screenshot:

Experiment 3 - C Program

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

int isKeyword(char buffer[])
{
    char keywords[32][10] = {"main", "auto", "break", "case", "char", "const", "continue", "default",
    "do", "double", "else", "enum", "extern", "float", "for", "goto",
    "if", "int", "long", "register", "return", "short", "signed",
    "sizeof", "static", "struct", "switch", "typedef",
    "unsigned", "void", "printf", "while"};
    int i, flag = 0;
    for(i = 0; i < 32; ++i)
    {
        if(strcmp(keywords[i], buffer) == 0)
        {
            flag = 1;
            break;
        }
    }
    return flag;
}

int main()
{
    char ch, buffer[15], operators[] = "+-*/%=";
    FILE *fp;
    int i, j = 0;
    fp = stdin;
    while((ch = fgetc(fp)) != EOF)
    {
        for(i = 0; i < 6; ++i)
        {
            if(ch == operators[i])
                printf("%c is operator\n", ch);
        }
        if(isalnum(ch))
        {
            buffer[j++] = ch;
        }
        else if((ch == ' ' || ch == '\n') && (j != 0))
        {
            buffer[j] = '\0';
            j = 0;
            if(isKeyword(buffer) == 1)
                printf("%s is keyword\n", buffer);
            else
                printf("%s is identifier\n", buffer);
        }
    }
    return 0;
}

```

Input:

```
main ( )  
{  
int a, b, c ; c  
= b + c;  
printf ( "%d" ,c ) ;  
}
```

Output:

```
main is keyword  
int is keyword  
a is indentifier  
b is indentifier  
c is indentifier  
c is indentifier  
= is operator  
b is indentifier  
+ is operator  
c is indentifier  
printf is keyword  
% is operator  
d is indentifier  
c is indentifier
```

Output Screenshot:

Experiment 3 - Output

```
main is keyword  
int is keyword  
a is indentifier  
b is indentifier  
c is indentifier  
c is indentifier  
= is operator  
b is indentifier  
+ is operator  
c is indentifier  
printf is keyword  
% is operator  
d is indentifier  
c is indentifier
```

Experiment No. 4

Question:

Design a lexical Analyzer to validate operators to recognize the operators +,-,*,/ using regular arithmetic operators using C

Code:

```
#include<stdio.h>
#include<conio.h>

int main()
{
    char s[5];
    printf("\n Enter any operator:");
    gets(s);
    switch(s[0])
    {
        case '>':
            if(s[1]=='=')
                printf("\n Greater than or equal");
            else
                printf("\n Greater than");
            break;
        case '<':
            if(s[1]=='=')
                printf("\n Less than or equal");
            else
                printf("\n Less than");
            break;
        case '=':
            if(s[1]=='=')
                printf("\n Equal to");
            else
                printf("\n Assignment");
            break;
        case '!':
            if(s[1]=='=')
                printf("\n Not Equal");
            else
                printf("\n Bit Not");
            break;
        case '&':
            if(s[1]=='&')
                printf("\n Logical AND");
            else
                printf("\n Bitwise AND");
            break;
        case '|':
            if(s[1]=='|')
                printf("\n Logical OR");
            else
                printf("\n Bitwise OR");
            break;
        case '+':
            printf("\n Addition");
```

```
        break;
    case '-':
        printf("\nSubstraction");
        break;
    case '*':
        printf("\nMultiplication");
        break;
    case '/':
        printf("\nDivision");
        break;
    case '%':
        printf("Modulus");
        break;
    default:
        printf("\n Not a operator");
    }
}
```

Code Screenshot:

Experiment 4 - C Program

```
#include<stdio.h>
#include<conio.h>

int main()
{
    char s[5];
    printf("\n Enter any operator:");
    gets(s);
    switch(s[0])
    {
        case '>':
            if(s[1]=='=')
                printf("\n Greater than or equal");
            else
                printf("\n Greater than");
            break;
        case '<':
            if(s[1]=='=')
                printf("\n Less than or equal");
            else
                printf("\n Less than");
            break;
        case '=':
            if(s[1]=='=')
                printf("\n Equal to");
            else
                printf("\n Assignment");
            break;
        case '!':
            if(s[1]=='=')
                printf("\n Not Equal");
            else
                printf("\n Bit Not");
            break;
        case '&':
            if(s[1]=='&')
                printf("\n Logical AND");
            else
                printf("\n Bitwise AND");
            break;
        case '|':
            if(s[1]=='|')
                printf("\n Logical OR");
            else
                printf("\n Bitwise OR");
            break;
        case '+':
            printf("\n Addition");
            break;
        case '-':
            printf("\n Subtraction");
            break;
        case '*':
            printf("\n Multiplication");
            break;
        case '/':
            printf("\n Division");
            break;
        case '%':
            printf("\n Modulus");
            break;
        default:
            printf("\n Not a operator");
    }
}
```

Input:

<=

Output:

Enter any operator:<=
Less than or equal

Output Screenshot:

Experiment 4 - Output

Enter any operator:<=
Less than or equal